

OWASP JUICE SHOP

Web Application Security Assessment

Professional Penetration Testing Report

Prepared by: John Osariemen Okosun

Cybersecurity Enthusiast | Web Developer

Assessment Target: OWASP Juice Shop 20.2.0

Environment: Local Docker Laboratory — 127.0.0.1:3000

Assessment Type: Authorized Security Assessment

Confidential — Training / Portfolio Use

Table of Contents

1. Executive Summary
2. Assessment Scope
3. Assessment Methodology
4. Risk Summary
5. Reconnaissance and Security Observations
6. Detailed Security Findings
7. Remediation Summary
8. Final Conclusion
9. Evidence and Screenshot Index
10. Appendices

1. Executive Summary

This assessment evaluated the OWASP Juice Shop web application in an authorized local laboratory environment. Testing combined reconnaissance, HTTP/API enumeration, authentication and authorization testing, controlled exploitation, and source-code review.

Six security findings were confirmed: SQL Injection in product search; SQL Injection in login; sensitive information exposure in JWT claims; broken function-level authorization with excessive user data exposure; Server-Side Request Forgery (SSRF); and Broken Object Level Authorization (BOLA/IDOR).

The assessment was limited to the local application at 127.0.0.1:3000. No production systems or unrelated external systems were targeted.

2. Assessment Scope

Target: OWASP Juice Shop

Application version: 20.2.0

Target URL: http://127.0.0.1:3000

Environment: Kali Linux, Docker, Firefox

Assessment type: Authorized web application security assessment

- Identify exposed application functionality and API endpoints.
- Evaluate authentication and authorization controls.
- Test input handling and injection resistance.
- Assess server-side request functionality.
- Review relevant source code to validate observed behavior.
- Collect evidence suitable for a professional security report.

3. Assessment Methodology

Reconnaissance

- Application identification using WhatWeb and HTTP responses.
- Review of response headers and exposed application information.

API Enumeration

- Identification and testing of public and protected API endpoints.
- Comparison of unauthenticated and authenticated behavior.

Authentication and Authorization

- Testing authentication boundaries and role enforcement.
- Testing unauthorized access to other users' data and objects.

Input Validation

- Controlled testing of product search and login inputs for SQL injection behavior.

SSRF Testing

- Controlled server-side request test against a tester-controlled local service.

Source-Code Review

- Review of relevant routes, middleware, models, and security functions to confirm root causes.

Evidence and Validation

- Screenshots and HTTP responses were collected.
- Modified test data was restored after the BOLA/IDOR test.

4. Risk Summary

Finding	Severity	Primary Classification
SQL Injection — Product Search	High	CWE-89 / OWASP A03:2021
SQL Injection — Login	High	CWE-89 / OWASP A03:2021
Sensitive Information Exposure in JWT	High	CWE-200 / OWASP A02:2021
Broken Function-Level Authorization / Excessive Data Exposure	High	CWE-862, CWE-200 / OWASP A01:2021
Server-Side Request Forgery	High	CWE-918 / OWASP A10:2021
Broken Object Level Authorization (BOLA/IDOR)	High	CWE-639 / OWASP API1:2023

5. Reconnaissance and Security Observations

WhatWeb identified the application as OWASP Juice Shop and returned HTTP 200. Observed response headers included X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN. Access-Control-Allow-Origin: * was also observed; this was treated as an observation rather than a confirmed vulnerability because sensitive cross-origin access was not established.

API enumeration identified public endpoints such as Products and Feedback, an unauthenticated whoami endpoint returning an empty user object, and protected endpoints such as /api/Users returning 401 when unauthenticated.

The application version endpoint returned 20.2.0 and the application-configuration endpoint returned configuration data. No exposed password, API-key, database credential, or private-key values were identified in the reviewed configuration response.

Evidence: Screenshots #3–#12 and #34.

6.1 Finding 1 — SQL Injection in Product Search

Description: User-controlled search input is incorporated directly into a raw SQL statement.

Technical Details: Route: /rest/products/search?q=... . Source review of /juice-shop/routes/search.ts showed the q parameter being concatenated into a SELECT statement. Controlled testing produced differing application behavior when SQL expressions were supplied, and source review confirmed unsafe SQL construction.

Impact: The database query can potentially be manipulated by an attacker, affecting query logic and potentially exposing or modifying database information depending on database permissions and reachable functionality.

Root Cause: Raw SQL is constructed with untrusted input instead of using parameterized queries.

Evidence: #13 13-product-search-baseline.png; #14 14-product-search-empty.png; #15 15-product-search-no-results.png; #19 19-sql-injection-source-code.png.

Remediation: Replace string-concatenated SQL with parameterized queries/prepared statements. Apply server-side validation and add regression tests for injection payloads.

References: CWE-89; OWASP Top 10 A03:2021 — Injection.

```
(kali㉿kali)-[~]
$ curl -i "http://127.0.0.1:3000/rest/products/search?q=juice"
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 12518
ETag: W/"30e6-9Lmvzq5s+ZOKZUfg1xx2FuCve1c"
Vary: Accept-Encoding
Date: Fri, 11 Sep 2026 20:41:29 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
(kali㉿kali)-[~]
$ curl -i "http://127.0.0.1:3000/rest/products/search?q="
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 16563
ETag: W/"40b3-6eE4aGFbh5haKBvUzP+TTAn7200"
Vary: Accept-Encoding
Date: Fri, 11 Sep 2026 20:47:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
(kali㉿kali)-[~]
$ curl -i "http://127.0.0.1:3000/rest/products/search?q=xyznotfound123"
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 30
ETag: W/"1e-JkPcI+pGj7BBTx0uZTVVIm91zaY"
Vary: Accept-Encoding
Date: Fri, 11 Sep 2026 20:55:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":[]}
```

```

(kali@kali)~$ sudo docker exec juice-shop /nodejs/bin/node -e "const fs=require('fs'); console.log(fs.readFileSync('/juice-shop/routes/search.ts','utf8'))"
/*
 * Copyright (c) 2014-2020 Björn Kimminich & the OWASP Juice Shop contributors.
 * SPDX-License-Identifier: MIT
 */

import { type Request, type Response, type NextFunction } from 'express'

import * as utils from '../lib/utils'
import * as models from '../models/index'
import { UserModel } from '../models/user'
import { challenges } from '../data/datacache'
import * as challengeUtils from '../lib/challengeUtils'

class ErrorWithParent extends Error {
  parent: Error | undefined
}

// vuln-code-snippet start unionSqlInjectionChallenge dbSchemaChallenge
export function searchProducts () {
  return (req: Request, res: Response, next: NextFunction) => {
    let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
    criteria = (criteria.length < 200) ? criteria : criteria.substr(0, 200)
    models.sequelize.query('SELECT * FROM Products WHERE ((name LIKE "%${criteria}%" OR description LIKE "%${criteria}%" AND deletedAt IS NULL) ORDER BY name') // vuln-code-snippet vuln-line unionSqlInjectionChallenge dbSchemaChallenge
    .then((products): any) => {
      const dataString = JSON.stringify(products)
      if (challengeUtils.notSolved(challenges.unionSqlInjectionChallenge)) { // vuln-code-snippet hide-start
        let solved = true
        UserModel.findAll().then(data => {
          const users = utils.queryResultToJson(data)
          if (users.data?.length) {
            for (let i = 0; i < users.data.length; i++) {
              solved = solved && utils.containsOrEscaped(dataString, users.data[i].email) && dataString.includes(users.data[i].password)
              if (!solved) {
                break
              }
            }
            if (solved) {
              challengeUtils.solve(challenges.unionSqlInjectionChallenge)
            }
          }
        }).catch((error: Error) => {
          next(error)
        })
      }
      if (challengeUtils.notSolved(challenges.dbSchemaChallenge)) {
        let solved = true
        void models.sequelize.query('SELECT sql FROM sqlite_master').then(([data]: any) => {
          const tableDefinitions = utils.queryResultToJson(data)

```

6.2 Finding 2 — SQL Injection in Login

Description: User-controlled login input is incorporated into a dynamically constructed SQL query.

Technical Details: Route: /rest/user/login. Source review showed email and password values being inserted into a Users SELECT statement. A malformed email input also caused a database error response, demonstrating unsafe query handling and verbose error behavior.

Impact: The authentication query may be manipulated, potentially affecting authentication logic. Verbose database errors can expose implementation details.

Root Cause: Dynamic SQL construction with user-controlled values and insufficient error abstraction.

Evidence: #24 24-login-source-code-analysis.png; #25 25-login-sql-injection-error.png.

Remediation: Use parameterized queries. Return generic authentication errors to clients and keep detailed database errors in server-side logs.

References: CWE-89; OWASP Top 10 A03:2021 — Injection.

```

// vuln-code-snippet start loginAdminChallenge loginBenderChallenge loginJimChallenge
export function login () {
  function afterLogin (user: User, res: Response, next: NextFunction) {
    verifyPostLoginChallenges(user) // vuln-code-snippet hide-line
    BasketModel.findOrCreate({ where: { UserId: user.id } })
      .then((basket): [BasketModel, boolean]) => {
        const authenticatedUser = { data: user, bid: basket.id } // keep track of original basket
        const token = security.authorize(authenticatedUser)
        security.authenticatedUsers.put(token, authenticatedUser)
        res.json({ authentication: { token, bid: basket.id, email: user.email } })
      }).catch((error: Error) => {
        next(error)
      })
  }

  return (req: Request, res: Response, next: NextFunction) => {
    verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
    models.sequelize.query('SELECT * FROM Users WHERE email = '${req.body.email}%' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL', { model: UserModel, plain: true }) // vuln-code-snippet vuln-line
    loginAdminChallenge loginBenderChallenge loginJimChallenge
    .then(authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJimChallenge
      const user = utils.queryResultToJson(authenticatedUser)
      if (user.data?.id && user.data.totpSecret !== '') {
        res.status(401).json({
          status: 'totp_token_required',
          data: {
            tnpToken: security.authorize({
              userId: user.data.id,
              type: 'password_valid_needs_second_factor_token'
            })
          }
        })
      } else if (user.data?.id) {
        afterLogin(user.data, res, next)
      } else {
        res.status(401).send(res.__(('Invalid email or password.')))
      }
    }).catch((error: Error) => {
      next(error)
    })
  }
}

```


References: CWE-200; OWASP Top 10 A02:2021 — Cryptographic Failures.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://127.0.0.1:3000/rest/user/login \
-H "Content-Type: application/json" \
-d '{"email":"j@wint0x.com","password":"admin123"}'
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 784
ETag: W/"310-Lp9E0tdJ00jLDyiF9Nb5Qw8mc4s"
Vary: Accept-Encoding
Date: Mon, 14 Sep 2026 15:18:05 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"authentication":{"token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJkYXRhIjpbInRZW40IiwiaWF0IjoxNjg5MDEyOTY5LmN5cCIsImF1dG8iOiJhbnVudG8iLCJpcmljaW50IjoiYSNkaW50OShNCmNkaW50NDoyMiQgKzAwOjAwIiwiaWZlZGVzZXRLZEF0IjpudWxsfnSwiYm9oL21xT2w22ibumX5NoteFltxMc1mpfo7fldotghs","bid":6,"umail":"","info.com"}]}
```

```
[kali@kali]~$ echo 'eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpb7mlkIjoyNSwidXNlcm5hbWUwIiOiIiLCJlbWFWbWpCI6ImpvaG5AaW5mby5jb20iLCJwYXNzd2YyIjAw5JCI6ImM4CwIjAiLCJwcm9maWwLSW1hZ2UwIiEvYXNzX2RXLzB1YmxpYy9pbWFnZXNmdXN5b2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZW9yZXQwIiOiIiLCJpc0FjZiIjOnN0YmIyMTQ0ZGZAwOjAwiIiwidGkgZXRLEZEFOiJpdWxsSWImlkjoZ2JlCjYXQyOjA5IjE3ODk0ZTk0W0DZ9.BHlti6TDGQIaFF3fMtlUGF3W5EzCzSfJsSQZxtFqqF1txMc1mpfo7f7ldotghs' | cut -d '.' -f2 | base64 -d 2>/dev/null
data":{"id":25,"username":"","email":"","password":"","token":"2026-09-14 14:54:22.214 +00:00","role":"customer","deletedAt":null,"active":true,"createdAt":"2026-09-14 14:54:22.214 +00:00","updatedAt":"2026-09-14 14:54:22.214 +00:00","deletedAt":null},"bid":6,
```

6.4 Finding 4 — Broken Function-Level Authorization and Excessive User Data Exposure

Description: An authenticated ordinary user was able to access /api/Users and retrieve records belonging to multiple accounts.

Technical Details: The endpoint requires authentication through `isAuthorized()`, but the reviewed authorization logic validates authentication without enforcing the required administrative role. Source review also showed broad serialization of user records.

Impact: This can expose information belonging to other users and may provide access to functionality intended for administrative users.

Root Cause: Authentication was enforced without sufficient role-level authorization, and the endpoint returned more user information than necessary.

Evidence: #10 10-users-api-unauthorized.png; #26 26-authenticated-users-api-authorization.png; #27 27-users-api-source-code.png; #28 28-isAuthorized-source-code.png.

Remediation: Restrict the endpoint to authorized administrative roles. Apply server-side role checks and return only fields required for the requested function.

References: CWE-862 and CWE-200; OWASP Top 10 A01:2021 — Broken Access Control.

```
(kali㉿kali)-[~/Desktop/Juice-shop-project/cors-test]
$ curl -i http://127.0.0.1:3000/api/Users
HTTP/1.1 401 Unauthorized
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: text/html; charset=utf-8
Vary: Accept-Encoding
Date: Fri, 11 Sep 2026 20:02:52 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked
```

Filter properties

JSON Raw

```
status: "success"
data: (23)[{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, ...]
  0: {id: 1, email: "...", role: "admin", ...}
  1: {id: 2, email: "...", role: "customer", ...}
  2: {id: 3, email: "...", role: "customer", ...}
  3: {id: 4, username: "...", email: "bjoern.kimminich@gmail.com", ...}
  4: {id: 5, email: "...", role: "deluxe", ...}
  5: {id: 6, email: "...", role: "admin", ...}
  6: {id: 7, email: "...", role: "customer", ...}
  7: {id: 8, email: "...", role: "customer", ...}
  8: {id: 9, email: "...", role: "admin", ...}
  9: {id: 10, username: "wurstbrot", email: "...", ...}
  10: {id: 11, email: "...", role: "customer", ...}
  11: {id: 12, email: "...", role: "admin", ...}
  12: {id: 13, email: "...", role: "deluxe", ...}
  13: {id: 15, email: "...", role: "accounting", ...}
  14: {id: 16, email: "...", role: "customer", ...}
  15: {id: 17, email: "demo", role: "customer", ...}
  16: {id: 18, username: "jOhNny", email: "...", ...}
  17: {id: 19, username: "F=ma3", email: "...", ...}
```

```

===== authenticatedUsers.ts =====
/*
 * Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
 * SPDX-License-Identifier: MIT
 */
import { type Request, type Response, type NextFunction } from 'express'
import { UserModel } from '../models/user'
import { decode } from 'jsonwebtoken'
import * as security from '../lib/insecurity'

async function retrieveUserList (req: Request, res: Response, next: NextFunction) {
  try {
    const users = await UserModel.findAll()

    res.json({
      status: 'success',
      data: users.map((user) => {
        const userToken = security.authenticatedUsers.tokenOf(user)
        let lastLoginTime: number | null = null
        if (userToken) {
          const parsedToken = decode(userToken, { json: true })
          lastLoginTime = parsedToken ? Math.floor(new Date(parsedToken?.iat ?? 0 * 1000).getTime()) : null
        }

        return {
          ...user.dataValues,
          password: user.password?.replace(/./g, '*'),
          totpSecret: user.totpSecret?.replace(/./g, '*'),
          lastLoginTime
        }
      })
    })
  } catch (error) {
    next(error)
  }
}

export default () => retrieveUserList

```

```

===== isAuthorized =====
ResponseWithUser | undefined
updateFrom: (req: Request, user: ResponseWithUser) => any
}

export const hash = (data: string) => crypto.createHash('md5').update(data).digest('hex')
export const hmac = (data: string) => crypto.createHmac('sha256', 'pa4qacea4VK9t9nGv7yZtwmj').update(data).digest('hex')

export const cutOffPoisonNullByte = (str: string) => {
  const nullByte = '%00'
  if (str.includes(nullByte)) {
    return str.substring(0, str.indexOf(nullByte))
  }
  return str
}

export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)
export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })
export const verify = (token: string) => token ? (jwt.verify as ((token: string, secret: string) => boolean))(token, publicKey) : false
export const decode = (token: string) => { return jwt.decode(token)?.payload }

export const sanitizeHtml = (html: string) => sanitizeHtmlLib(html)
export const sanitizeLegacy = (input = '') => input.replace(/<(?:\w+)\W+?[\w]/gi, '')
export const sanitizeFilename = (filename: string) => sanitizeFilenameLib(filename)
export const sanitizeSecure = (html: string): string => {
  const sanitized = sanitizeHtml(html)
  if (sanitized === html) {
    return html
  } else {
    return sanitizeSecure(sanitized)
  }
}

export const authenticatedUsers: IAuthenticatedUsers = {
  tokenMap: {},
  idMap: {},
  put: function (token: string, user: ResponseWithUser) {
    this.tokenMap[token] = user
    this.idMap[user.data.id] = token
  },
  get: function (token?: string) {
    return token ? this.tokenMap[utils.unquote

```

6.5 Finding 5 — Server-Side Request Forgery (SSRF)

Description: The profile image URL functionality can cause the application server to fetch a user-supplied URL.

Technical Details: Route: POST /profile/image/url. Source review showed imageUrl being passed to fetch(). A controlled test server on the Kali host received a request originating from the Juice Shop Docker environment and returned the expected test content.

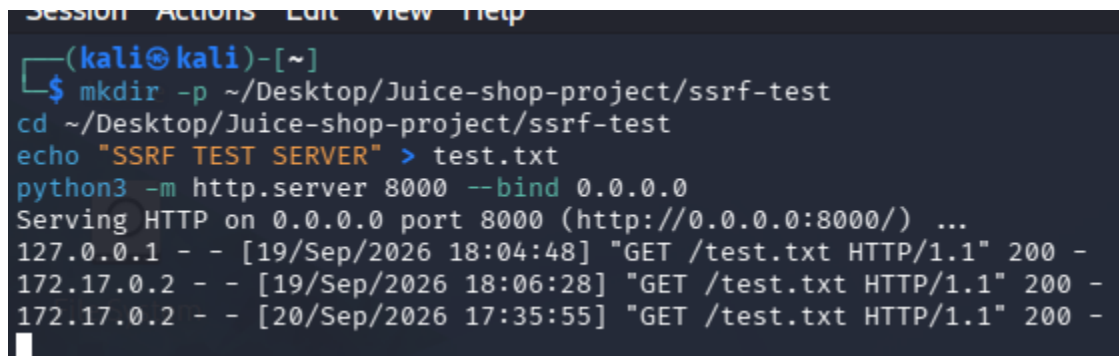
Impact: An attacker may be able to make the application server send requests to destinations the attacker cannot directly access. Cloud metadata or sensitive internal services were not demonstrated.

Root Cause: The server performs outbound requests to user-controlled URLs without sufficiently restrictive destination validation and network controls.

Evidence: #31 31-ssrf-server-side-request-proof.png.

Remediation: Use an allowlist where possible, restrict protocols, block loopback/private/link-local destinations when not required, and apply network-level egress controls.

References: CWE-918; OWASP Top 10 A10:2021 — Server-Side Request Forgery.

A terminal window on a Kali Linux machine showing the setup and execution of an SSRF test. The user creates a directory, sets up a test file, and runs a Python HTTP server. The server logs show three GET requests to /test.txt from different IP addresses, all returning a 200 status code.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/Desktop/Juice-shop-project/ssrf-test
cd ~/Desktop/Juice-shop-project/ssrf-test
echo "SSRF TEST SERVER" > test.txt
python3 -m http.server 8000 --bind 0.0.0.0
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [19/Sep/2026 18:04:48] "GET /test.txt HTTP/1.1" 200 -
172.17.0.2 - - [19/Sep/2026 18:06:28] "GET /test.txt HTTP/1.1" 200 -
172.17.0.2 - - [20/Sep/2026 17:35:55] "GET /test.txt HTTP/1.1" 200 -
```

6.6 Finding 6 — Broken Object Level Authorization (BOLA/IDOR)

Description: An authenticated user was able to read and modify a BasketItem object belonging to another user's basket by referencing its object ID.

Technical Details: User 25 accessed BasketItem ID 1, which belonged to Basket 1/User 1. GET returned the object successfully. A PUT changing only quantity also succeeded. The original quantity was restored after validation.

Impact: This permits unauthorized access to and modification of another user's object.

Root Cause: Object-level ownership was not consistently checked before the generated CRUD operation processed the requested BasketItem ID.

Evidence: #35 35-basketitem-idor-unauthorized-read.png; #36 36-basketitem-idor-unauthorized-modification.png.

Remediation: Verify object ownership or explicit authorization for every read and write operation. Add cross-user authorization tests for object identifiers.

References: CWE-639; OWASP API Security Top 10 API1:2023 — Broken Object Level Authorization.

New Request	Search	Blocking	Status	Met...	Domain	File	Initiator	Type	Transferred	Size	Headers	Cookies	Request	Response
GET	http://127.0.0.1:3000/api/BasketItems/1		200	GET	127.0.0.1:3000	eggfruit_juice.jpg	img	jpeg	12.35 kB	11.92 kB	Filter properties			
URL Parameters			200	GET	127.0.0.1:3000	elderflower_cordial.jpg	img	jpeg	51.03 kB	50.60 kB	JSON			
name	value		200	GET	127.0.0.1:3000	fruit_press.jpg	img	jpeg	13.53 kB	13.10 kB	status: "success"			
Host	127.0.0.1:3000		200	GET	127.0.0.1:3000	grape_juice.png	img	png	14.33 kB	13.90 kB	data: { ProductId: 1, BasketId: 1, id: 1, ... }			
Accept-Encoding	gzip, deflate, br, zstd		200	GET	127.0.0.1:3000	green_smoothie.jpg	img	jpeg	12.70 kB	12.26 kB	ProductId: 1			
Referer	http://127.0.0.1:3000/		200	GET	127.0.0.1:3000	permafrost.jpg	img	jpeg	84.07 kB	93.64 kB	BasketId: 1			
Content-Length	44		200	GET	127.0.0.1:3000	6	polyfills.js.2 (fetch)	json	539 B	154 B	id: 1			
Origin	http://127.0.0.1:3000		200	GET	127.0.0.1:3000	51Hd+Tue Sep 22 2026	polyfills.js.2 (fetch)	json	665 B	279 B	quantity: 2			
Connection	keep-alive		200	GET	127.0.0.1:3000	1	polyfills.js.2 (fetch)	json	932 B	546 B	createdAt: "2026-09-16T13:38:12.827Z"			
			200	GET	127.0.0.1:3000	1	NetUtilSys.mik14...	json	539 B	154 B	updatedAt: "2026-09-16T13:38:12.827Z"			

```

JSON
status: "success"
data: { ProductId: 1, BasketId: 1, id: 1, ... }
  ProductId: 1
  BasketId: 1
  id: 1
  quantity: 3
  createdAt: "2026-09-16T13:38:12.827Z"
  updatedAt: "2026-09-28T09:48:29.197Z"

```

7. Remediation Summary

Area	Recommended Action
SQL Injection	Use parameterized queries/prepared statements throughout the application; avoid concatenating user input into SQL.
JWT / Authentication	Use minimal JWT claims; keep sensitive information server-side; review token lifecycle and signing-key protection.
Authorization	Separate authentication from authorization; enforce role and object ownership checks server-side.
Data Minimization	Return only fields required for each API operation and avoid broad user-record serialization.
SSRF	Validate URLs, allowlist destinations where feasible, block restricted networks, and enforce egress controls.
Error Handling	Return safe client-facing errors while retaining detailed diagnostics in server-side logs.
Security Testing	Add automated tests covering injection, role authorization, BOLA/IDOR, data exposure, and SSRF controls.

8. Final Conclusion

The assessment confirmed six security findings across input handling, authentication, authorization, API data exposure, token design, and server-side network request processing.

The most important remediation themes are parameterized database queries, strong server-side authorization, data minimization, secure JWT design, strict server-side URL validation, and object-level access control.

Testing was limited to the authorized local OWASP Juice Shop laboratory environment. No production systems or unrelated external systems were targeted. A follow-up assessment should be performed after remediation to verify that the findings have been resolved.

9. Evidence and Screenshot Index

#	Filename / Reference	Purpose
#3	03-http-response-headers.png	HTTP response headers and security

		observations.
#4	04-api-endpoint-enumeration.png	API endpoint enumeration.
#5	05-feedback-api-response.png	Feedback API response.
#6	06-cors-test-server.png	Controlled CORS test server.
#7A	07A-browser-cors-proof.png	Browser CORS behavior.
#7B	07B-cors-test-server.png	CORS test-server evidence.
#8	08-whoami-unauthenticated-response.png	Unauthenticated whoami response.
#9	09-browser-whoami-cors-test.png	Browser whoami/CORS evidence.
#10	10-users-api-unauthorized.png	Unauthenticated Users API request.
#11	11-api-route-enumeration.png	API route enumeration.
#12	12-products-api-response.png	Products API response.
#13/#14/#15	13-product-search-baseline.png; 14-product-search-empty.png; 15-product-search-no-results.png	Product-search testing evidence.
#19	19-sql-injection-source-code.png	Product-search SQL injection source evidence.
#22/#23	JWT evidence — exact filenames not preserved in assessment notes.	JWT evidence.
#24	24-login-source-code-analysis.png	Login SQL source evidence.
#25	25-login-sql-injection-error.png	Login database error evidence.
#26	26-authenticated-users-api-authorization.png	Authenticated ordinary-user access to Users API.
#27	27-users-api-source-code.png	Users API source evidence.
#28	28-isAuthorized-source-code.png	Authorization middleware source evidence.
#31	31-ssrf-server-side-request-proof.png	Controlled SSRF request proof.
#34	34-admin-configuration-unauthenticated.png	Unauthenticated configuration endpoint observation.
#35	35-basketitem-idor-unauthorized-read.png	Unauthorized BasketItem read.
#36	36-basketitem-idor-unauthorized-modification.png	Unauthorized BasketItem modification.

10. Appendices

Appendix A — Testing Environment

- Kali Linux virtual machine
- Docker container running OWASP Juice Shop
- Firefox browser
- WhatWeb
- cURL
- Browser Developer Tools
- Node.js/JavaScript utilities for controlled validation
- Juice Shop source-code review inside the Docker container

Appendix B — Evidence Handling

- Review screenshots before publication.
- Redact passwords, tokens, cookies, JWTs, API keys, and other credentials.
- Do not include live authentication secrets in the final report.
- Restore modified test data where practical; the BasketItem quantity used in the BOLA/IDOR test was restored.

Appendix C — Reporting Limitations

The assessment was performed against a deliberately vulnerable training application. Findings and observed behavior should therefore be interpreted as laboratory evidence rather than an assessment of a production deployment. Where impact was not directly demonstrated, the report states the relevant testing limitation.